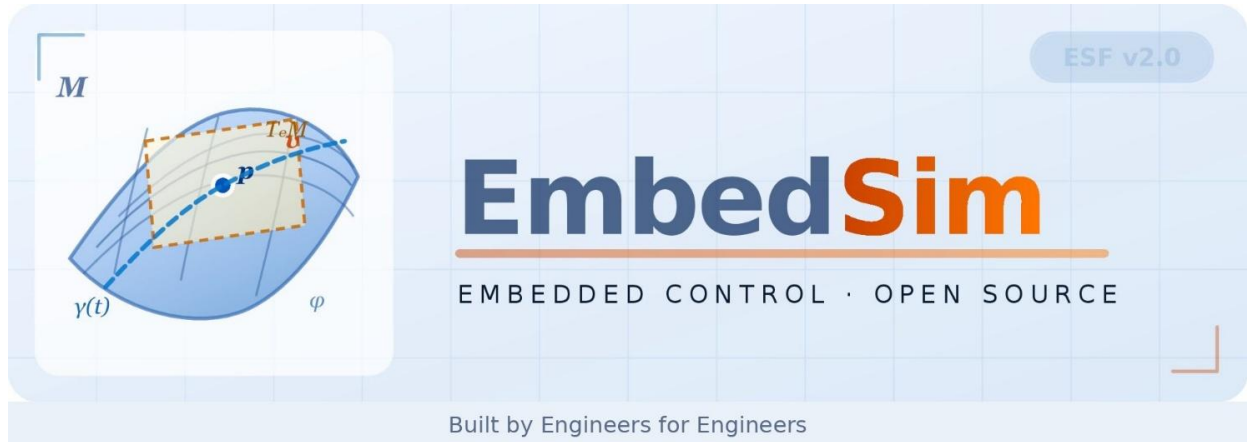




EmbedSim Framework

Technical Architecture & Algorithm Reference

Open-Source Python · Modelica · C | Control-Systems Simulation

Targets: Infineon AURIX TriCore • ARM Cortex-M • FMI 2.0 Co-Simulation



github.com/vectorsim/embed_sim_project

Table of Contents

Overture — A Walking Tour of EmbedSim	3
Act I — The Problem Worth Solving	3
Act II — The Plant: Modelica to FMU to Python Block	3
Act III — Wire and Experiment in Python (No C Yet)	3
Step 1 — Wire the closed loop	3
Step 2 — Experiment freely	3
Act IV — Write <code>compute_c()</code> and Verify Against Python	3
The VectorBlock skeleton	3
A concrete example: LowPassFilter	3
Step 3 — Verify C matches Python	3
Act V — Generate MISRA-C and Deploy	3
The Complete Workflow at a Glance	3
Abstract	4
1 Introduction	5
2 Formal Graph Model	6
2.1 Block Diagram as a Directed Graph	6
2.2 Signal Type: VectorSignal	6
2.3 Block Categories	6
3 Dependency Resolution: Topological Sort via DFS	7
3.1 The Ordering Problem	7
3.2 Two-Pass Algorithm	7
3.3 Complexity and Correctness	7
4 Numerical Integration of Dynamic States	8
4.1 Problem Statement	8
4.2 Forward Euler — <code>ODESolver.EULER</code>	8
4.3 Runge-Kutta 4 — <code>ODESolver.RK4</code>	8
4.4 Solver Selection Guide	8
5 Dual-Backend Execution Architecture	9
5.1 Design Rationale	9
5.2 Backend Dispatcher	9
5.3 Python Backend	9
5.4 Cython / C Backend	9
5.5 PYXInspector and Auto-Population	9
5.6 C Code Generation	9
6 Feedback Loop Breaking	10
6.1 The Algebraic Loop Problem	10
6.2 The LoopBreaker Interface	10
6.3 VectorDelay — Example	10
7 Signal Type System and Precision	11
7.1 Default Precision: float32	11
7.2 Per-Block Precision Override	11
7.3 Signal Dimension Conventions	11

8 Simulation Lifecycle	12
8.1 Construction Phase	12
8.2 Run Phase	12
8.3 Signal Recording: VectorScope	12
9 Architecture Summary	13
10 References	14

Overture — A Walking Tour of EmbedSim

Before the formal mathematics begins, let us walk the entire workflow from first idea to deployed C code — exactly as a composer sketches themes before the orchestra enters. Every concept introduced here reappears in the later sections with full rigour. Read this chapter once, in order, and the rest of the document will feel familiar.

Act I — The Problem Worth Solving

You are designing a motor controller for an industrial pump. You have a Modelica model of the motor, a PI speed loop in mind, and eventually need deterministic C code running on an Infineon AURIX TriCore at 20 kHz. The standard route costs five-figure licence fees. EmbedSim is the open-source route.

The journey unfolds in four acts, always in this order:

The EmbedSim Workflow — Four Acts, One Direction

- ① Model your plant in Modelica (OpenModelica) → export FMU
- ② Wrap, wire, and experiment entirely in Python — no C yet, no compilation
- ③ Write `compute_c()`, run it alongside `compute_py()`, verify they agree
- ④ Generate MISRA-C with LoopGenerator → flash to your MCU

The golden rule: Python comes first. Always. C is only introduced once the Python simulation is correct and the gains are tuned.

Act II — The Plant: Modelica to FMU to Python Block

Open OpenModelica and create a simple DC motor with back-EMF:

```
// DCMotor.mo — OpenModelica source
model DCMotor
  parameter Real R = 1.0 "Armature resistance [ $\Omega$ ]";
  parameter Real L = 0.01 "Armature inductance [H]";
  parameter Real Km = 0.1 "Back-EMF / torque constant";
  parameter Real J = 0.01 "Rotor inertia [ $\text{kg}\cdot\text{m}^2$ ]";
  parameter Real b = 0.01 "Viscous friction [ $\text{N}\cdot\text{m}\cdot\text{s}/\text{rad}$ ]";
  Real i(start=0) "Armature current [A]";
  Real omega(start=0) "Angular velocity [ $\text{rad}/\text{s}$ ]";
  input Real V_in "Supply voltage [V]";
  output Real speed = omega;
equation
  L*der(i) = V_in - R*i - Km*omega;
  J*der(omega) = Km*i - b*omega;
end DCMotor;
```

Export from OpenModelica: Simulation → Export FMU → FMI 2.0 Co-Simulation. You receive `DCMotor.fmu`. Wrapping it in EmbedSim takes one line:

```
from embedsim.fmu_block import FMUBlock

motor = FMUBlock(
    fmu_path = 'DCMotor.fmu',
    input_names = ['V_in'], # voltage command → FMU input
    output_names = ['speed'], # angular velocity ← FMU output
    dt = 1e-4, # 10 kHz step
```

```

    name          = 'DCMotor',
)

```

What is an FMU?

A Functional Mock-up Unit (FMU) is a ZIP archive containing a compiled shared library (.dll on Windows, .so on Linux) and an XML model description. EmbedSim uses FMPy to load and step it at each simulation timestep. The Modelica differential equations run inside the FMU's own solver — EmbedSim just calls `step()` and reads back outputs.

Act III — Wire and Experiment in Python (No C Yet)

Step 1 — Wire the closed loop

Add a reference signal and a PI controller, then wire everything together with the `>>` operator:

```

from embedsim.core_blocks import VectorStep, VectorSum, VectorEnd
from embedsim.pid_block import PIBlock
from embedsim.sim_engine import EmbedSim, ODESolver
from embedsim.vector_scope import VectorScope

# — Blocks —————
ref = VectorStep(step_time=0.1, initial=0.0, final=100.0, name='ref')
motor = FMUBlock('DCMotor.fmu', ['V_in'], ['speed'], dt=1e-4)
error = VectorSum(signs=[+1, -1], name='error') # ref - speed
ctrl = PIBlock(Kp=5.0, Ki=50.0, name='PI')
end = VectorEnd(name='out')

# — Wiring —————
ref >> error          # port 0: reference
motor >> error         # port 1: measured speed (feedback)
error >> ctrl >> motor # PI output → voltage → motor
motor >> end           # record speed

# — Simulate —————
scope = VectorScope()
scope.add(motor, label='speed_rad_s')
scope.add(ctrl, label='voltage_V')

sim = EmbedSim(sinks=[end], T=1.0, dt=1e-4, solver=ODESolver.RK4)
sim.run(scope)
scope.plot()

```

The >> Operator Explained

`a >> b` connects the output `VectorSignal` of block `a` to the next available input of block `b`.
`a >> b >> c` chains: (`a` feeds `b`) then (`b` feeds `c`) — Python left-to-right.
 For multi-input blocks (`VectorSum`), call `>>` once per port, in port order.
 Returns `b` so chains compose across multiple lines without temporary variables.

Step 2 — Experiment freely

This is your laboratory. Every change takes effect instantly — nothing is compiled, nothing relinked:

What to try	How	What you learn
Tune Kp, Ki	PIBlock(Kp=8.0, Ki=80.0)	Faster rise, more overshoot?
Change solver	ODESolver.EULER vs RK4	Speed vs accuracy trade-off

Inject sensor noise	GaussianNoiseBlock(sigma=0.5) >> error	Noise sensitivity of PI gains
Raise sample rate	dt=5e-5 (20 kHz)	Does aliasing matter here?
Saturate output	VectorSaturation(± 24 V) after ctrl	Simulate voltage rail limit
Swap plant	StateSpaceBlock instead of FMUBlock	Analytical vs Modelica model
Add measurement delay	VectorDelay(dt=2e-4) on feedback	Sensor latency effect

Keep iterating here until the simulation matches your design targets. Only then proceed to Act IV.

Act IV — Write compute_c() and Verify Against Python

Once the Python simulation is correct, you implement compute_c() for each custom block. The structure of every VectorBlock makes this straightforward:

The VectorBlock skeleton

```
class VectorBlock:
    # — Attributes (identity, wiring, control) —————
    name: str # display label
    inputs: list[VectorBlock] # upstream blocks
    output: VectorSignal # current output (float32 array)
    is_dynamic: bool # True → participates in ODE integration
    use_c_backend: bool # False by default; flip to True for C path

    # — C code-generation metadata (auto-filled by PYXInspector) —
    PYX_FILE: str # path to .pyx Cython wrapper
    C_SOURCES: list[str] # .c files to compile
    C_HEADERS: list[str] # .h files to #include
    C_INPUT_MAP: dict # maps port index → signal name

    # — The two methods you implement —————
    def compute_py(self, t, dt, input_values) -> VectorSignal: ...
    def compute_c(self, t, dt, input_values) -> VectorSignal: ...

    # — Dispatcher — never overridden —————
    def compute(self, t, dt, input_values):
        if self.use_c_backend:
            return self.compute_c(t, dt, input_values)
        return self.compute_py(t, dt, input_values)
```

A concrete example: LowPassFilter

We build a first-order IIR low-pass filter — compute_py first, compute_c second.

```
# — 1. Python backend: implement and test this first —————
import numpy as np
from embedsim.core_blocks import VectorBlock, VectorSignal

class LowPassFilter(VectorBlock):
    """y[k] =  $\alpha \cdot u[k] + (1-\alpha) \cdot y[k-1]$ ,  $\alpha = 2\pi \cdot f_c \cdot dt / (1 + 2\pi \cdot f_c \cdot dt)$ """
```

```

def __init__(self, fc: float, dt: float, name='LPF'):
    super().__init__(name=name)
    self.alpha = 2*np.pi*fc*dt / (1 + 2*np.pi*fc*dt)
    self.y_prev = np.float32(0.0)

def compute_py(self, t, dt, input_values):
    u = input_values[0].value[0]
    y = self.alpha * u + (1.0 - self.alpha) * self.y_prev
    self.y_prev = y
    self.output = VectorSignal(np.array([y], dtype=np.float32))
    return self.output

```

Run the simulation. Plot it. Satisfy yourself that the filter cutoff is correct. Now write the C:

```

/* low_pass_filter.c - MISRA-C:2012 */
#include "low_pass_filter.h"
static float32_t s_y_prev = 0.0F;

void LPF_step(float32_t u, float32_t alpha, float32_t * const y_out)
{
    *y_out = (alpha * u) + ((1.0F - alpha) * s_y_prev);
    s_y_prev = *y_out;
}

# low_pass_filter.pyx - Cython wrapper (GIL released during C call)
# distutils: sources = [low_pass_filter.c]
cdef extern from "low_pass_filter.h":
    void LPF_step(float u, float alpha, float *y_out)

def step(float u, float alpha):
    cdef float y_out
    with nogil:
        LPF_step(u, alpha, &y_out)
    return y_out

# — 2. Add compute_c() to the same class —————
PYX_FILE = 'low_pass_filter.pyx' # PYXInspector auto-fills metadata

def compute_c(self, t, dt, input_values):
    import low_pass_filter as _lpf
    u = float(input_values[0].value[0])
    y = _lpf.step(u, float(self.alpha))
    self.output = VectorSignal(np.array([y], dtype=np.float32))
    return self.output

```

Step 3 — Verify C matches Python

Run both backends on identical inputs and assert they agree:

```

# — Verification harness —————
lpf_py = LowPassFilter(fc=500.0, dt=1e-4, name='LPF_py')
lpf_c = LowPassFilter(fc=500.0, dt=1e-4, name='LPF_c')
lpf_c.use_c_backend = True

test_signal = np.random.randn(1000).astype(np.float32)
results_py, results_c = [], []

for u_val in test_signal:
    sig = [VectorSignal(np.array([u_val]))]
    results_py.append(lpf_py.compute_py(0, 1e-4, sig).value[0])
    results_c.append(lpf_c.compute_c(0, 1e-4, sig).value[0])

assert np.allclose(results_py, results_c, atol=1e-5), 'C and Python disagree!'
print('✓ C backend matches Python to within 1e-5')

```

```
# Now flip the whole simulation to C and re-run:
ctrl.use_c_backend = True
lpf_c.use_c_backend = True
sim.run(scope)      # same diagram, same wiring - C path throughout
```

The Verification Principle

compute_py() is always the ground truth. It never changes once validated.
compute_c() must produce bit-for-bit equivalent results (within float32 tolerance).
Run the full closed-loop simulation with use_c_backend=True as a system-level check.
If Python and C agree on the simulation, the C code is ready for the MCU.

Act V — Generate MISRA-C and Deploy

With the C backends verified, trigger code generation. EmbedSim traverses the topologically sorted block list and emits a complete, allocation-free step function:

Stage	What happens	Output
1. Topology sort	DFS resolves block execution order	Ordered block list
2. PYXInspector	Reads .pyx files, extracts port counts & names	Block metadata
3. LoopGenerator	_emit_block() for each block in topo order	C function body
4. File emit	Writes embedsim_loop.c and embedsim_loop.h	embedsim_gen/ folder

```
from embedsim.code_generator import CodeGenStart, CodeGenEnd, LoopGenerator

cg_start = CodeGenStart(name='cg_start')
cg_end   = CodeGenEnd   (name='cg_end')

cg_start >> ref >> error >> ctrl >> lpf_c >> motor >> cg_end

gen = LoopGenerator(sinks=[cg_end], output_dir='embedsim_gen/')
gen.generate_loop()
# ✓ embedsim_gen/embedsim_loop.c
# ✓ embedsim_gen/embedsim_loop.h
```

The generated function is a fixed-step, no-malloc, MISRA-C:2012 compliant loop body — paste it directly into your AURIX or Cortex-M project:

```
/* embedsim_loop.c - AUTO-GENERATED by EmbedSim. DO NOT EDIT. */
/* MISRA C:2012 compliant. No heap. No recursion. No globals. */
#include "embedsim_loop.h"
#include "pi_block.h"
#include "low_pass_filter.h"
#include "dc_motor_fmu_c.h" /* OpenModelica C export */

void EmbedSim_Step(void)
{
    float32_t sig_ref;
    float32_t sig_error;
    float32_t sig_ctrl_out;
```



```

float32_t sig_lpf_out;
float32_t sig_motor_speed;

Step_compute(&sig_ref);
sig_error = sig_ref - sig_motor_speed; /* feedback: prev step */
PI_step(sig_error, &sig_ctrl_out);
LPF_step(sig_ctrl_out, LPF_ALPHA, &sig_lpf_out);
DCMotor_step(sig_lpf_out, &sig_motor_speed);
}

```

FMU → C for MCU deployment

On the MCU, replace FMUBlock with the C sources exported by OpenModelica (Simulation → Generate C Code). The C_CUSTOM_EMIT attribute on FMUBlock lets you inject the Modelica-generated function call verbatim into the generated embedsim_loop.c — no hand-editing of generated files required.

The Complete Workflow at a Glance

Step	Tool / Action	Artefact
Design plant	OpenModelica (.mo)	DCMotor.mo
Export FMU	Simulation → Export FMU	DCMotor.fmu
Wrap plant	FMUBlock (one line)	motor block
Wire controller	>> operator in Python	simulation.py
Experiment & tune	compute_py() — no compilation	Plots, scope data, final gains
Write C & verify	compute_c() + assert allclose	Validated C functions
Generate MCU loop	LoopGenerator.generate_loop()	embedsim_gen/*.c / *.h
Build & flash	TASKING / gcc-arm / icc	Firmware binary

The Python simulation and the deployed C firmware execute the same algorithm, verified to agree before deployment. Discrepancies between simulation and hardware are minimised by design — not by luck.

What comes next in this document

Section 2 — Formal graph model: the mathematics of VectorBlock, VectorSignal, and directed-graph semantics.

Section 3 — Dependency resolution: the two-pass DFS topological sort and loop-breaker detection.

Section 4 — Numerical integration: Forward Euler and RK4, stability regions, and solver selection.

Section 5 — Dual-backend architecture: PYXInspector, auto-population, and C code generation.

Section 6 — Feedback loops: the LoopBreaker mixin, VectorDelay, and algebraic loop detection.

Section 7 — Signal type system: float32 / float64, dimension conventions, and MCU alignment.

Abstract

EmbedSim is an open-source, Python-native, Modelica block-diagram simulation framework designed as an affordable, engineering-grade alternative to MATLAB/Simulink for embedded control-systems development. The framework unifies a pure-Python simulation path with a compiled Cython/C execution backend, enabling control algorithms to be written once in C and validated in Python before deployment to 32-bit MCU targets such as the Infineon AURIX TriCore TC3xx/TC4xx and ARM Cortex-M families.

This document is a formal technical reference. It describes the mathematical foundations, computational graph model, dependency-resolution algorithms (depth-first search with loop-breaker semantics), ODE integration strategies (Forward Euler, Runge-Kutta 4), dual-backend execution architecture, and the C code-generation pipeline. The treatment is intentionally rigorous and is intended for engineers and researchers who need a precise understanding of the system's internals.

1 Introduction

Modern embedded control design demands tools that span the full V-model: from high-fidelity numerical simulation, through rapid-prototyping on hardware-in-the-loop rigs, to production-quality C code that satisfies MISRA C:2012 and IEC 26262 ASIL-D requirements. Commercial tools address this need at considerable licence cost. EmbedSim addresses the same workflow in the open-source ecosystem.

The framework is structured around three orthogonal concerns:

- Graph model: a directed, acyclic (after loop breaking) block diagram whose nodes are computational units and whose edges carry typed vector signals.
- Execution semantics: a topologically sorted forward-evaluation pass at every discrete timestep, with algebraic feedback loops resolved by causal delay elements.
- Numerical integration: a choice of first-order (Euler) or fourth-order (RK4) ODE solvers applied to all dynamic (stateful) nodes simultaneously.

Section 2 establishes notation and the formal graph model. Section 3 derives the dependency-resolution algorithm. Section 4 covers ODE integration. Section 5 describes the dual-backend architecture and C code generation. Section 6 addresses the feedback loop-breaking mechanism. Section 7 documents the signal type system.

2 Formal Graph Model

2.1 Block Diagram as a Directed Graph

A simulation is modelled as a directed graph $G = (V, E)$ where each vertex $v \in V$ is a `VectorBlock` instance and each directed edge $(u, v) \in E$ represents a signal flowing from the output port of u to one input port of v . Edges are stored implicitly: each block maintains an ordered list `v.inputs` of its upstream predecessors.

Formally, the output of block v at time t is a function of its input signals and, for dynamic blocks, an internal state vector:

```
Static block:   $y_v(t) = f_v(y_{u1}(t), y_{u2}(t), \dots)$ 
Dynamic block:  $y_v(t) = g_v(x_v(t), y_{u1}(t), \dots)$ 
                $dx_v/dt = h_v(x_v(t), y_{u1}(t), \dots)$ 
```

The distinction between static and dynamic nodes is encoded in the boolean attribute `VectorBlock.is_dynamic`. Only dynamic blocks participate in the ODE integration step.

2.2 Signal Type: VectorSignal

Every edge carries a `VectorSignal`, a thin wrapper around a NumPy ndarray. The framework defaults to float32 (single-precision IEEE 754) for MCU compatibility, but individual blocks may request float64 via the dtype constructor argument.

Attribute	Description
value	np.ndarray — the signal vector (float32 by default)
name	Optional string label for plotting and debugging
shape	Derived from value.shape; scalar signals have shape (1,)

2.3 Block Categories

Category	Description & Examples
Source	No inputs; generate signals as functions of time only. <code>VectorStep</code> , <code>VectorConstant</code> , <code>VectorSine</code> , <code>GaussianNoiseBlock</code> , <code>FMUBlock</code> (as plant).
Static processing	One or more inputs, no internal state. Output depends only on current inputs. <code>VectorGain</code> , <code>VectorSum</code> , <code>VectorSaturation</code> , <code>VectorProduct</code> .
Dynamic	One or more inputs, internal state vector $x(t)$. Output depends on inputs and state. <code>VectorIntegrator</code> , <code>StateSpaceBlock</code> , <code>TransferFunctionBlock</code> , <code>SpeedPIBlock</code> , <code>FMUBlock</code> .
Sink	No downstream consumers; record outputs. <code>VectorEnd</code> captures history. Also <code>VectorScope</code> for arbitrary blocks.

3 Dependency Resolution: Topological Sort via DFS

3.1 The Ordering Problem

For a forward evaluation pass to be correct, each block must be computed only after all blocks that feed into it have already produced their outputs for the current timestep. This is the classical topological ordering problem on a DAG. In a pure feed-forward diagram with no cycles, standard post-order DFS from the sink nodes yields a valid execution order in $O(|V| + |E|)$ time. Feedback loops introduce directed cycles; these must be broken before topological sorting is possible.

3.2 Two-Pass Algorithm

EmbedSim resolves this in two passes implemented in `traverse_blocks_from_sinks_with_loops()`:

- **Pass 1 — Loop-Breaker Discovery.** A full DFS from every sink, visiting all reachable blocks, collects the set `L` of `LoopBreaker` instances. This pass is cycle-safe because it does not attempt to build an ordering; it simply records membership.
- **Pass 2 — Ordered DFS.** A second DFS builds the execution order. When traversing the input edges of a block, if an input block belongs to `L` it is inserted immediately (without recursing into its own inputs). This cuts every feedback edge, making the residual graph a DAG and allowing the DFS to complete without infinite recursion.

```
function TRAVERSE(sinks):
    L ← ∅                                // loop-breaker set
    visited ← ∅
    order ← []
    // Pass 1: find all LoopBreaker nodes
    for each sink s:
        FIND_BREAKERS(s, L, visited)
    visited ← ∅
    in_stack ← ∅                          // cycle detection
    // Pass 2: post-order DFS producing execution order
    for each sink s:
        DFS(s, L, visited, in_stack, order)
    return order

function DFS(v, L, visited, in_stack, order):
    if v ∈ visited: return
    if v ∈ in_stack: raise AlgebraicLoopError(v)
    in_stack ← in_stack ∪ {v}
    for each u ∈ v.inputs:
        if u ∈ L:                          // cut feedback edge
            if u ∉ visited:
                visited ← visited ∪ {u}
                order.append(u)
        else:
            DFS(u, L, visited, in_stack, order)
    in_stack ← in_stack \ {v}
    visited ← visited ∪ {v}
    order.append(v)                       // post-order: v after all inputs
```

3.3 Complexity and Correctness

Time complexity: $O(|V| + |E|)$ for both passes combined. Space complexity: $O(|V|)$ for the `visited` and `in_stack` sets, plus $O(|V|)$ for the output list. The post-order property guarantees that for every non-loop-

breaking edge (u, v) , u appears before v in order. True algebraic loop detection raises a `ValueError` naming the offending block, prompting the engineer to insert a `VectorDelay`.

4 Numerical Integration of Dynamic States

4.1 Problem Statement

Let the complete system state be the concatenation of all dynamic block states: $X(t) = [x_1(t); x_2(t); \dots; x_n(t)] \in \mathbb{R}^N$. The coupled ODE system is $dX/dt = F(X(t), U(t), t)$ where $U(t)$ is the aggregate of all source block outputs at time t . EmbedSim solves this with fixed-step methods over the interval $[0, T]$ with step size dt .

4.2 Forward Euler — ODESolver.EULER

```
X(t + dt) = X(t) + dt * F( X(t), U(t), t )

# Implementation:
for each dynamic block b:
    u          = gather_inputs(b)
    b.derivative = b.get_derivative(t, u)    # dx/dt = f(x,u,t)
    b.state     += dt * b.derivative        # Euler step
```

Global truncation error: $O(dt)$. Appropriate for non-stiff systems where speed dominates accuracy requirements.

4.3 Runge-Kutta 4 — ODESolver.RK4

```
k1 = F( X(t), U(t), t )
k2 = F( X(t)+dt/2*k1, U(t+dt/2), t+dt/2 )
k3 = F( X(t)+dt/2*k2, U(t+dt/2), t+dt/2 )
k4 = F( X(t)+dt*k3, U(t+dt), t+dt )

X(t+dt) = X(t) + (dt/6) * (k1 + 2*k2 + 2*k3 + k4)
```

Global truncation error: $O(dt^4)$. RK4 is unconditionally the recommended solver for closed-loop control systems where integrator accuracy directly affects controller convergence.

4.4 Solver Selection Guide

Scenario	Recommended Solver
Fast real-time preview, non-stiff plant	EULER
Closed-loop control, integrators present	RK4
PMSM FOC with SMC / SMO	RK4
Stiff ODE (eigenvalue ratio > 1000)	RK4 with reduced dt
LQR state-space, linear plant	RK4 or EULER
Pure discrete-time system (no integrators)	EULER
FMU co-simulation plant	EULER (FMU has internal solver)

5 Dual-Backend Execution Architecture

5.1 Design Rationale

EmbedSim targets two distinct deployment environments that impose conflicting constraints. In simulation mode (Python), rapid iteration speed, NumPy broadcasting, and Matplotlib integration are paramount. In embedded mode (MCU), deterministic execution time, minimal RAM, and MISRA C:2012 compliance are mandatory. The dual-backend architecture resolves this tension by allowing the same block-diagram description to drive either runtime.

5.2 Backend Dispatcher

```
def compute(self, t, dt, input_values):
    if self.use_c_backend:
        return self.compute_c(t, dt, input_values)
    return self.compute_py(t, dt, input_values)
```

Subclasses override `compute_py()` (required) and `compute_c()` (optional). The dispatcher is never overridden. This separation allows a developer to write and verify the Python path first, then introduce the C path incrementally, with the Python path always available as a reference.

5.3 Python Backend

The Python backend is the primary development path. It uses NumPy array operations throughout, giving vectorised performance without requiring compilation. All standard EmbedSim blocks ship with complete Python implementations. The Python backend is also the regression baseline: automated tests run the Python path and compare outputs against reference trajectories.

5.4 Cython / C Backend

The C backend enables zero-overhead execution of the hot simulation path. A Cython wrapper (`.pyx`) exposes the C function via a Python-compatible interface with the GIL released during computation. The wrapper receives inputs as a flat float32 array, calls the C function, and returns outputs as another flat float32 array.

```
def compute_c(self, t, dt, input_values):
    u = np.empty(N_INPUTS, dtype=np.float32)
    u[0] = input_values[0].value[0]    # I_alpha
    u[1] = input_values[0].value[1]    # I_beta
    u[2] = input_values[1].value[0]    # theta_r
    self._wrapper.set_inputs(u)
    self._wrapper.compute()            # GIL released
    y = self._wrapper.get_outputs()    # float32[N_OUT]
    self.output = VectorSignal(y, self.name)
    return self.output
```

Fallback: if the compiled extension is not present, EmbedSim catches the `ImportError` and transparently falls back to `compute_py()`. This means a simulation script runs correctly on any developer machine regardless of whether the Cython extensions have been built.

5.5 PYXInspector and Auto-Population

The PYXInspector module automates the boilerplate of declaring C code-generation metadata on block subclasses. When a subclass declares a PYX_FILE class attribute, the VectorBlock.__init_subclass__() hook fires and calls auto_populate_from_pyx(cls, pyx_file). PYXInspector parses the .pyx source and extracts NUM_INPUTS, OUTPUT_SIZE, C_SOURCES, and C_HEADERS.

```
# Minimal declaration – everything else is auto-populated by PYXInspector
class DCMotorBlock(VectorBlock):
    PYX_FILE = 'electrical_blocks/dc_motor.pyx'

# After class definition, auto-populated attributes are available:
# DCMotorBlock.NUM_INPUTS    → 1    (V_in)
# DCMotorBlock.OUTPUT_SIZE   → 1    (speed)
# DCMotorBlock.C_SOURCES     → ['dc_motor.c']
# DCMotorBlock.C_HEADERS     → ['dc_motor.h']
```

5.6 C Code Generation

The code_generator.py module traverses the topologically ordered block list and emits embedsim_loop.c and embedsim_loop.h into the project-root embedsim_gen/ directory. The generated .c file is a fixed-step, main-loop-friendly step function suitable for direct inclusion in an AURIX TriCore or Cortex-M4 bare-metal application.

6 Feedback Loop Breaking

6.1 The Algebraic Loop Problem

A feedback loop in a block diagram creates a directed cycle in G . If all blocks in the cycle are static (instantaneous), the cycle represents a true algebraic loop. EmbedSim detects these and raises a `ValueError`. If at least one block in the cycle is causal (its output at time t depends only on its input at time $t - dt$), the cycle can be broken with `VectorDelay`.

6.2 The LoopBreaker Interface

```
class LoopBreaker:
    """Mixin: block can supply output without needing current inputs."""
    def get_loop_breaking_output(self) -> VectorSignal:
        """Return previous-timestep output (or initial value at t=0)."""
        ...
```

6.3 VectorDelay — Example

```
# Feedback from motor output back to error sum:
# ref → [error] → [PI] → [motor] →
#           ↑           [VectorDelay] ←
#
from embedsim.core_blocks import VectorDelay

delay = VectorDelay(initial=np.array([0.0], dtype=np.float32),
                    name='fb_delay')

# Wire the feedback path:
motor >> delay
delay >> error # port 1: delayed speed measurement
```

Semantics: $y(t) = u(t - dt)$, $y(0) = \text{initial}$. The one-step delay introduces an approximation error of $O(dt)$ in the feedback path. For typical control-loop sample rates (10–20 kHz on AURIX), the phase lag is negligible at controller bandwidths well below the sample rate.

7 Signal Type System and Precision

7.1 Default Precision: float32

All VectorSignal objects default to numpy.float32. This mirrors the native word size of the target MCU (AURIX TriCore uses IEEE 754 single-precision in hardware). Using float32 throughout simulation ensures that precision-related artefacts visible in hardware are faithfully reproduced in the simulation. The global default is controlled by the module-level constant DEFAULT_DTYPE in core_blocks.py.

7.2 Per-Block Precision Override

```
# Per-block override: this integrator runs in double precision
integrator = VectorIntegrator(dtype=np.float64, name='high_prec')

# Mixed precision is supported: NumPy handles implicit casts
# float64 integrator → float32 gain → float32 motor signal
```

7.3 Signal Dimension Conventions

Signal type	NumPy shape	C equivalent
Scalar control signal	(1,)	float32_t
Alpha-beta (Clarke frame)	(2,)	float32_t[2]
Three-phase (abc frame)	(3,)	float32_t[3]
dq-frame vector	(2,)	float32_t[2]
State-space state	(n,)	float32_t[n]

8 Simulation Lifecycle

8.1 Construction Phase

When `EmbedSim(sinks, T, dt, solver)` is called: `traverse_blocks_from_sinks_with_loops(sinks)` is called, producing the full ordered block list. Blocks are classified into `dynamic_blocks`, `static_blocks`, and `loop_breakers`. `SimulationStats` is initialised with `loop_breakers_count` and `feedback_loops_count`.

8.2 Run Phase

```
for each block b: b.reset()           # clear all outputs
t = 0
for step in range(int(T / dt)):
    _compute_all_blocks(t)           # forward pass in topo order
    scope.record(t)                  # sample monitored signals
    if solver == RK4:
        _integrate_dynamics_rk4(t)   # 4-stage state update
    else:
        _integrate_dynamics_euler(t) # 1-stage state update
    t += dt
_compute_all_blocks(T)               # capture final state
scope.record(T)
```

Note the ordering: outputs are computed and recorded before states are advanced. This implements the standard zero-order-hold (ZOH) discretisation convention: the output at step k reflects the state at the beginning of step k , not the end.

8.3 Signal Recording: VectorScope

```
scope = VectorScope()
scope.add(motor, indices=[0], label='speed_rad_s')
scope.add(ctrl, indices=[0], label='voltage_V')

sim = EmbedSim(sinks=[end], T=1.0, dt=1e-4, solver=ODESolver.RK4)
sim.run(scope)

# Post-run: extract NumPy arrays for analysis
speed = scope.get_signal('speed_rad_s', index=0) # shape (10000,)
voltage = scope.get_signal('voltage_V', index=0)

import matplotlib.pyplot as plt
plt.plot(scope.time, speed, label='Speed [rad/s]')
plt.plot(scope.time, voltage, label='Voltage [V]')
plt.legend(); plt.grid(); plt.show()
```

9 Architecture Summary

Component	Role
VectorBlock	Abstract base; dual-backend dispatcher; DFS graph node
VectorSignal	Typed float32/64 vector; graph edge data carrier
FMUBlock	Wraps Modelica FMU via FMPy; full VectorBlock interface
traverse_blocks_from_sinks...	Two-pass DFS; topological sort; algebraic loop detection
LoopBreaker (mixin)	Marks causal delay blocks; cuts feedback cycles in DFS
VectorDelay	Canonical one-step delay; LoopBreaker implementation
EmbedSim	Orchestrator: construct, classify, run, record
VectorScope	Signal recorder; post-run NumPy array extraction
ODESolver	Enum: EULER ($O(dt)$), RK4 ($O(dt^4)$)
_integrate_dynamics_rk4	4-stage coupled RK4 across all dynamic blocks
CodeGenStart / LoopGenerator	C code emitter; produces embedsim_loop.c / .h
PYXInspector	Parses .pyx; auto-populates NUM_INPUTS, OUTPUT_SIZE etc.
TopologyPrinter	Console + browser SVG block diagram renderer with swim lanes

10 References

1. Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4th ed.). Johns Hopkins University Press.
2. Butcher, J. C. (2008). Numerical Methods for Ordinary Differential Equations (2nd ed.). Wiley.
3. AUTOSAR Consortium. (2023). AUTOSAR Classic Platform — Software Component Template. Release 22-11.
4. Infineon Technologies AG. (2022). AURIX TC3xx User Manual. Rev 2.0.
5. MISRA Consortium. (2012). MISRA C:2012 — Guidelines for the Use of the C Language in Critical Systems.
6. FMI Project. (2021). Functional Mock-up Interface Specification 2.0.4. Modelica Association.
7. Cython Documentation. (2024). Cython 3.x Reference Manual. cython.readthedocs.io.
8. Kahn, A. B. (1962). Topological sorting of large networks. Communications of the ACM, 5(11), 558–560.
9. Fritzson, P. (2015). Principles of Object-Oriented Modeling and Simulation with Modelica 3.3. Wiley-IEEE Press.

EmbedSim Framework · Open Source · MIT Licence
github.com/vectorsim/embed_sim_project